MeterIQ: Explainable Smart Meter Analytics Platform

Project Proposal for UCS503P
Submitted to: Mr. Jeelani
Thapar Institute of Engineering and Technology

Yashit Arora, CSED* Aarushi Gahlawat, CSED*

*Roll No: 1024160006, Email: yarora1_be24@thapar.edu
*Roll No: 1024160008, Email: agahlawat_be24@thapar.edu

September 1, 2026

1 Higher-order goal

MeterIQ aims to make smart-meter data easier to analyse and use. The system will identify unusual consumption behaviour and provide short-term load forecasts.

We will work with smart-meter datasets from Mathura and Bareilly. The main goal is to bring anomaly detection and load forecasting together in one platform so that a user can view a meter's consumption, investigate unusual behaviour and see its expected future load.

2 Time-to-value

- Understand and clean the Mathura and Bareilly datasets.
- Build simple baselines for anomaly detection and load forecasting.
- Compare suitable ML approaches using the prepared data.
- Connect the analysis with a backend, data storage and dashboard.
- Use Git and GitHub to track development.

3 Problem Statement

Smart meters generate a large amount of electricity data, but manually checking this data for unusual behaviour is difficult. Sudden changes in consumption may have different causes, and it is not practical to inspect every reading individually.

The available datasets contain timestamped consumption along with measurements such as voltage, current and frequency. The project aims to use these readings to identify unusual patterns and forecast future load in a way that is easier for a user to understand.

An anomaly will be treated as a signal for further investigation, not as automatic proof of electricity theft.

4 Proposed Solution

4.1 Overview

MeterIQ will be developed as a web-based smart-meter analytics platform with two main functions: anomaly detection and short-term load forecasting.

The anomaly module will identify unusual meter behaviour and provide an anomaly or risk indication. The forecasting module will estimate future electricity consumption from historical readings.

Both results will be available through the same application instead of being handled as separate analysis notebooks.

4.2 Core workflow

1. Load and inspect the smart-meter data.
2. Clean the data and prepare useful features.
3. Run anomaly detection and load forecasting.
4. Store and expose the results through the application.
5. Allow the user to search for a meter and view its history, alerts and forecast.

4.3 Operational constraints for an educational Software Engineering project

- The actual dataset will be studied before selecting features and models.
- Chronological splits will be used for forecasting to avoid data leakage.
- Anomaly results will be treated as investigation indicators.
- The system will be developed in separate modules so that the parts can be tested independently.

5 Solution Approach

The project will follow a simple workflow: understand the data, prepare it, build the ML components and then connect them to the application.

5.1 Data understanding and preprocessing

The Mathura and Bareilly datasets will be inspected separately. The available data includes timestamp, energy consumption, average voltage, average current, frequency and meter information.

Preprocessing will include timestamp handling, checking missing and duplicate records, checking unusual values and validating meter-wise data. Time-based, lag and rolling features will be created where useful.

5.2 Anomaly and risk detection

The anomaly module will look for consumption behaviour that differs from a meter's usual pattern. Voltage, current, frequency and time-based information may also be considered.

Models such as Isolation Forest and Local Outlier Factor will be compared. The final approach will be selected after testing it on the available data.

The output will include an anomaly indication and supporting information so that the user can understand why a reading was flagged.

5.3 Short-term load forecasting

The forecasting module will use historical consumption to estimate future load. A simple historical baseline will be used first and then compared with ML models such as Linear Regression and Random Forest.

Lagged consumption, rolling statistics and calendar features will be considered. Models will be evaluated using MAE, RMSE and MAPE on a chronological test period.

5.4 Explainability

MeterIQ will show more than an anomaly flag. For a flagged reading, the user will be able to see the consumption pattern and relevant changes that contributed to the alert.

The system will clearly separate model output from the information calculated from the data so that an anomaly is not presented as a confirmed cause.

5.5 Web architecture

The application will have a web-based frontend, a backend layer, data-processing and ML components, and a suitable data-storage solution.

Python, Pandas, NumPy and Scikit-learn will be used for the data and ML work. Git and GitHub will be used for version control.

5.6 Use Case Diagram

The Use Case Diagram represents the interactions between the users and the Smart Meter Analysis and Theft Detection System. It shows the major functionalities provided by the system and the relationships between the actors and these functionalities.

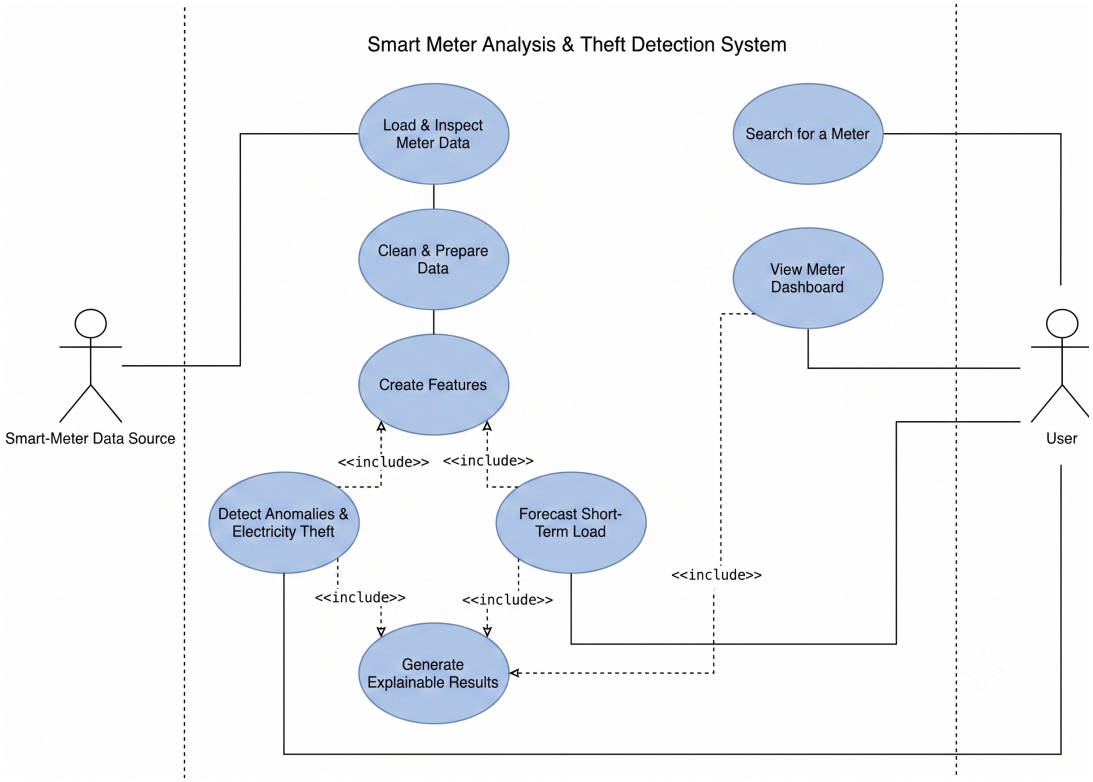

Figure 1: Use Case Diagram of Smart Meter Analysis and Theft Detection System

5.7 Data Flow Diagram

The Level-0 Data Flow Diagram (DFD) illustrates the overall flow of data between the Smart-Meter Data Source, the MeterIQ System, and the user. It provides a high-level representation of how meter data enters the system, how the system processes it, and how the results and user requests flow between the system and the distribution personnel.

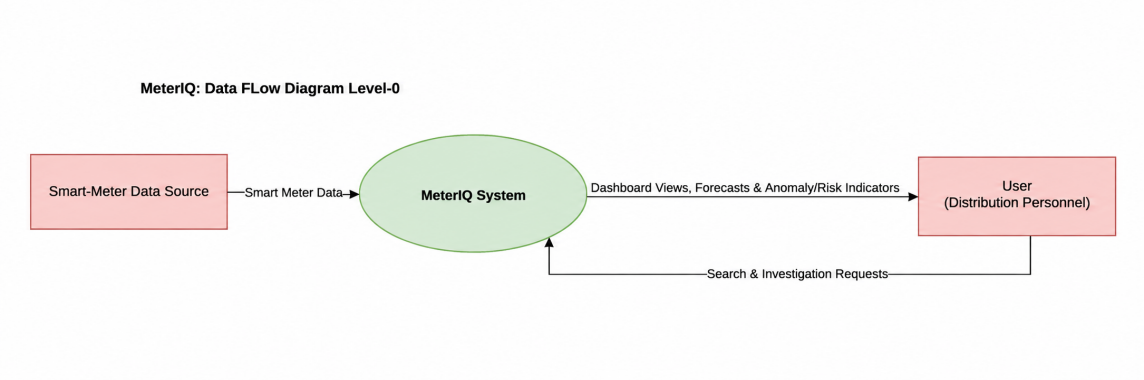

Figure 2: Level-0 Data Flow Diagram of the MeterIQ System

5.8 CI/CD and fast delivery enabler

GitHub will be used throughout development. Automated testing and basic build checks can be added as the application develops to make changes easier to test and manage.

6 Evaluation Criterion

6.1 Primary evaluation metric

For load forecasting, the main evaluation metrics will be:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- Mean Absolute Percentage Error (MAPE)

For anomaly detection, the detected patterns will be checked for consistency and usefulness. Confirmed theft labels will not be assumed unless they are available in the data.

6.2 Secondary evaluation metrics

The application will also be checked for correct data retrieval, working API calls, correct display of results, basic response time and integration between the ML and application components.

6.3 Pilot validation plan

Forecasting models will be tested chronologically so that future information is not used during training. Different models will be compared using the same evaluation period.

Anomaly results will be inspected at the meter level and compared with historical behaviour.

7 Scalability

The system will be designed so that additional meters and future readings can be added later. Keeping the data, ML and application parts separate will also make it easier to update the models when more data becomes available.

8 Engine availability heuristic

The availability of a meter will be checked using the completeness and continuity of its readings. Large gaps in the data may affect forecasting and anomaly analysis, so data availability will be considered before using a meter for detailed analysis.

9 Project scope and deliverables

9.1 Initial deliverable

The first deliverable will be a cleaned and documented version of the Mathura and Bareilly data with exploratory analysis of consumption and electrical behaviour.

9.2 Subsequent deliverables

- Data preprocessing and feature engineering pipeline.
- Anomaly detection module.
- Short-term load forecasting module.
- Model evaluation and comparison.
- Explainability for anomaly results.
- Backend and data-storage integration.
- Web dashboard for meter analysis.
- Testing and project documentation.

10 Risks and mitigations

- **Data quality:** missing, duplicate or unusual readings will be checked during preprocessing.
- **Different meter behaviour:** analysis will consider meter-level patterns instead of assuming all meters behave the same way.
- **False anomalies:** flagged readings will be treated as investigation signals rather than confirmed theft.
- **Data leakage:** chronological validation and carefully created lag features will be used.
- **Model selection:** simple baselines will be compared with ML models before choosing the final approach.
- **System integration:** the ML, backend and frontend components will be developed separately and integrated step by step.

11 Summary

MeterIQ will analyse smart-meter data from Mathura and Bareilly using two connected components: anomaly detection and short-term load forecasting.

The system will help a user view meter consumption, identify unusual behaviour and understand the information behind an alert. It will also provide future load estimates based on historical readings.

The final project will combine the data pipeline, ML models and web-based application into one system. The focus will be on useful and understandable results rather than making unsupported claims about electricity theft.